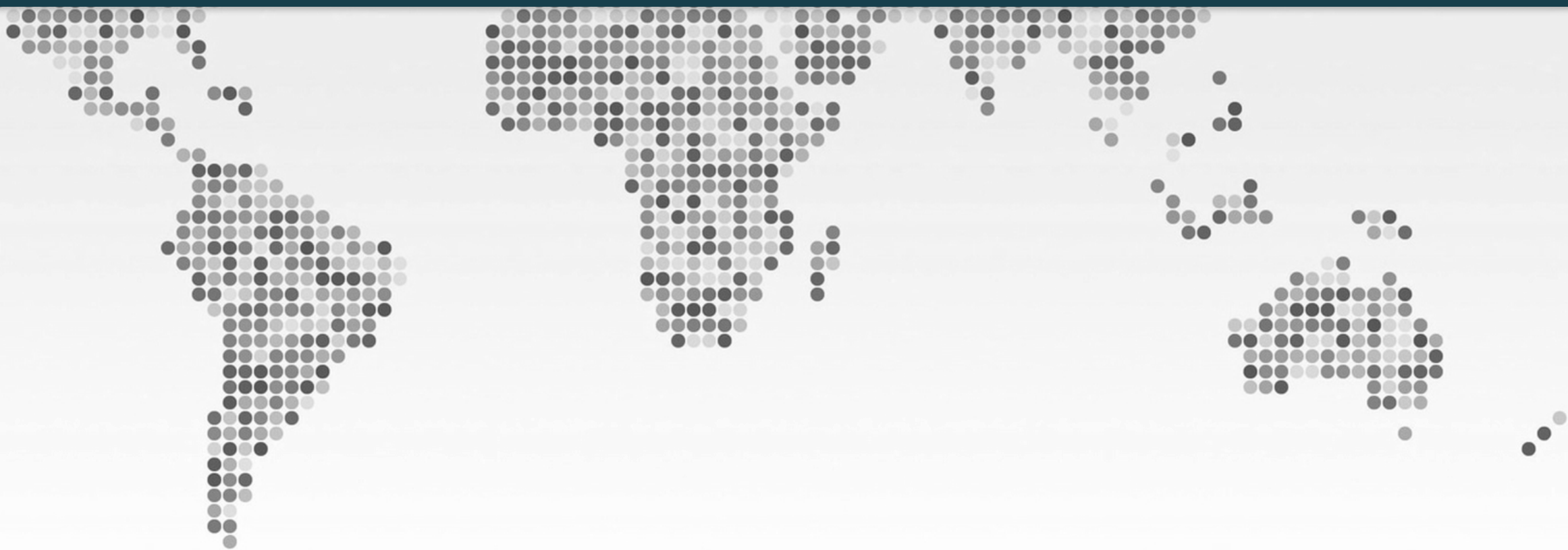


Software Engineering for Artificial Intelligence

Validation and Verification



Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems

It is surprisingly difficult to define what it means for an AI model to be “correct” or “buggy”. To explain why, it is necessary to first look at how we traditionally define and evaluate the correctness of software implementations.

When discussing correctness, we always need to ask, “*Correct with regard to what?*” In traditional software, we evaluate correctness by comparing the software's behavior against its intended behavior (the oracle).

Ideally, we have a **precise specification** of the intended behavior of a software component or a requirement of a system's behavior in the real world that can be used to determine whether a computed output for a given input is correct.

Verification. The process of establishing correctness by checking an implementation against a specification or requirement.

Validation. The process of checking whether the specification or implementation meets the customer's problem and needs.

The difference between verification and validation is important to understand the typical notion of correctness. For the sake of clarity, let us consider a non-AI example of computing the date following a provided date, i.e., a method called *next_date*.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems

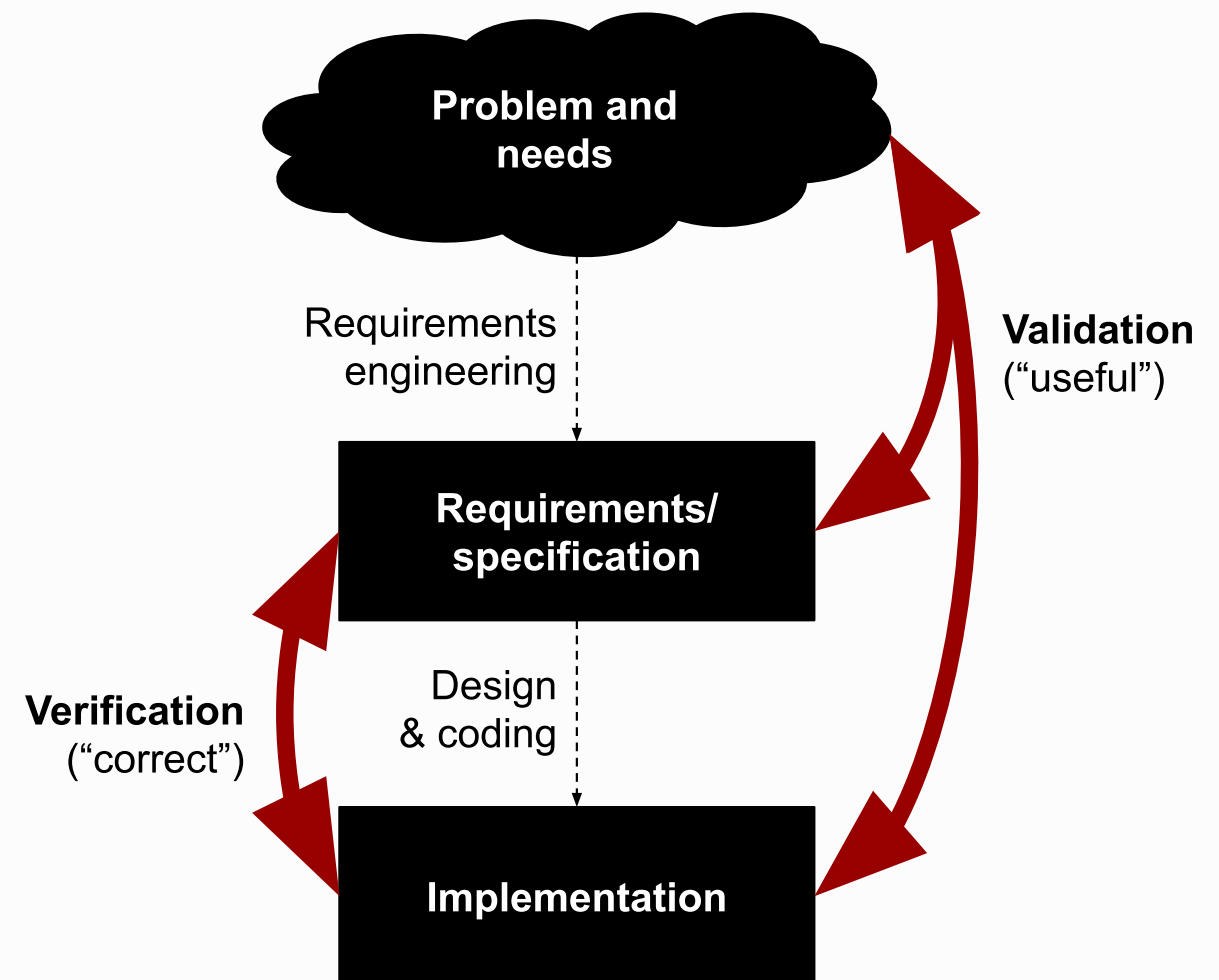
To *verify* that the implementation of *next_date* implements the specification for that problem **correctly**, we can write tests that check input-output pairs. If any test case fails, either the test was wrong, or we have found a bug.

To *validate* that the implementation **meets a customer's needs**, we might show the finished product to the customer. If the customer needed a *next_date* function in the Hebrew calendar instead, we simply implemented the wrong specification.

In that case, the implementation would still be *technically correct according to the provided specification*, but it would simply not be useful to the customer.

The correctness of software is primarily evaluated through **testing**. We can not exhaustively test all possible inputs, because there are simply too many.

So, while a good testing regime might give us some confidence in correctness, it will not provide correctness guarantees — we may miss some bugs in testing.



Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems

How we evaluate AI models differs fundamentally from traditional software testing. Generally speaking, AI is just an algorithm that takes inputs to compute an output; the key difference is that it has been learned from data rather than manually coded.

Again, the question is, “*Correct with regard to what?*”

For AI models, we have **no specification** that we could use to determine which outputs are correct for any given input. **After all, we use AI precisely because we do not have such specifications**, typically because the problem is too complex to specify or the rules are simply unknown.

We might have *ideas, goals, intuitions*, and *general descriptions*, for example “detect whether there is cancer in the image”, as well as examples of input-output pairs, which could be seen as partial or implicit specifications, but *nothing we can write down* as concrete rules as to when an output is correct for any given input.

In practice, we fundamentally accept that we cannot avoid some wrong predictions. When using AI, *we cannot and do not expect* all outputs to be correct, if we can even determine whether a prediction is correct in the first place. We would not write a test suite with individual input-output examples and expect all tests to pass. A single wrong prediction is not a reason to reject a model.

Overall, we have a **weak notion of correctness**. We evaluate models with examples, but not against specifications of what the model should generally do.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems

Note, that the difference here is **not** non-determinism or some notion of a probabilistic specification. Most ML models are deterministic during inference: the issue is the **lack of a specification, not statistical noise**.

So, let's be pragmatic: A better way to think about how to evaluate a model is to think about a model not as correct or wrong or buggy, but about whether *the model fits a problem or whether it is useful for a task*. The key question is whether the model's predictions, in the presence of occasional mistakes, are **good enough for practical use**.

Returning to our distinction between verification and validation in software engineering, we argue that evaluating model quality is ultimately a validation question rather than a verification question. Without a specification, we cannot establish correctness, but instead evaluate whether the model fits a user's needs.

As a consequence of the points above, we may even recommend avoiding terms like *model bug* and *correctness of a model*, because they do not easily match evaluating models where a single wrong prediction is not usually a problem in isolation. Instead, we recommend using terms that refer to the **fit** or **accuracy of a model**.

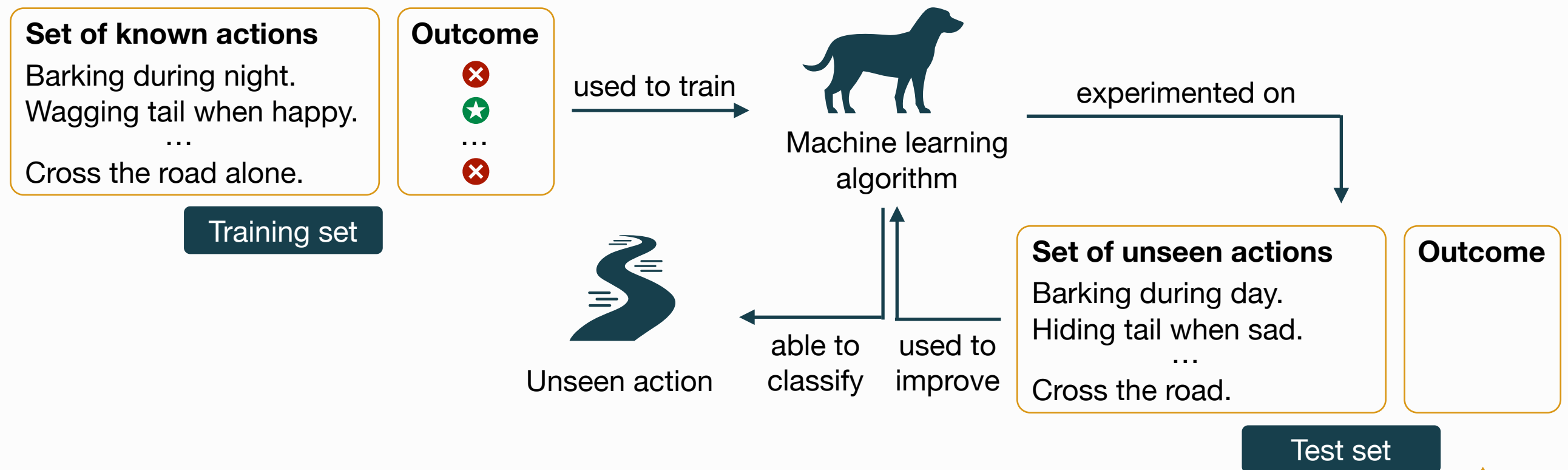
Precisely for the points mentioned above, we typically start evaluating AI algorithms - and ML models in particular, through **model validation**. In this case, we get some form of prediction accuracy on some test data, under the assumption that the test data will resemble real data.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Validation

Validation is the process of **estimating** whether the specification or implementation of the model will meet the customer's problem and needs.

This typically starts with a so-called *in-vitro* experimentation of the capabilities of artificial intelligence solutions, namely the definition of an empirical setting where the accuracy of a model is experimented.



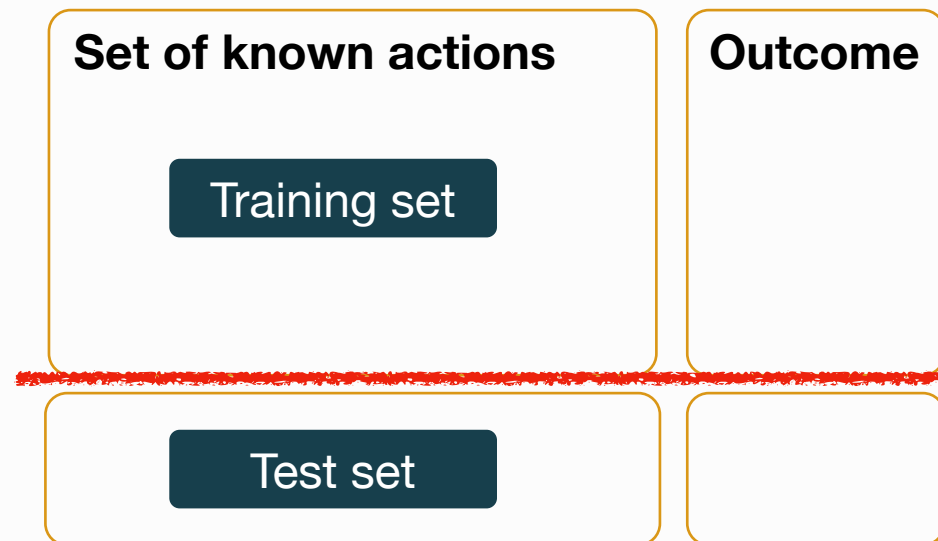
As labeling additional data may be complex, the general idea is to split the original dataset into two sets, so that part of that is used for training and the remaining part for testing. NB: You cannot use the entire dataset for both training and testing!

Be careful here: if we want to see how effective a machine learning is, we need an *oracle* reporting the actual labels for the examples in the test set. The oracle will be only used to compare the predictions done by the learner with the truth values.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Validation

Hence, we need to identify strategies to split the original dataset. The easiest, low-intensive-brain-activity method is the percentage split method.



Well, this is not really a low-intensive-brain-activity method. The choice of the percentage may have a strong influence the resulting performance —> if you train with less data, you lose prediction capabilities, but if you train with more data you lose in terms reliability of the evaluation!

As some of you might have heard already, the 70%-30% rule (70% training, 30% test) seems to be the most established one.

My empirical soul must say something here: there is **NO empirical study** showing that this is a good way to configure the validation procedure. In other terms, this rule emerged anecdotally, which is quite ironic: an empirical method to validate machine learning system has never been subject of empirical validation...

This sounds weird, and it is weird indeed. However, it comes with two main questions: (1) What is the right splitting percentage to use? (2) Who ensures that my validation is reasonable enough to consider it good?

Who knows?

No one!

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Validation

We need something else. As some of you might have heard already, cross-validation can come to rescue.

Set of known actions	Outcome

In cross-validation you split the dataset in n parts, called *folders*, and you validate the model using $n-1$ folders as training and the remaining folder as test. The process is then repeated so that each folder is used as test exactly once.

Typically, a 10-fold cross-validation is applied, with each folder defined randomly.

Sad but true, even the idea of using $n=10$ has never been empirically assessed. Nonetheless, it is way more credible because you experiment the model with (slightly) different training data and (slightly) different test data.

Cross-fold validation is the standard *de-facto* in model validation. However, someone might ask: when validating a model using this strategy, where are we going to preprocess data and configure hyper-parameters?

Do you remember the F1 driver parallel? Preprocessing and configuration are required to let the machine learner have the best possible environment to make predictions: yet, such a best possible environment should be assessed against a real case scenario which might diverge from the ideal case.

Hence, the test sets might be as real as possible!

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Validation

We need something else. As some of you might have heard already, cross-validation can come to rescue.

Set of known actions	Outcome
----------------------	---------

--	--

Each fold used as test set should be left untouched.

At each iteration, the training data must be subject of the preprocessing and configuration steps.

This is an additional reason why experimenting with machine learning systems takes a lot: in the validation phase, you need to run expensive computation multiple times!

And the story is not yet at the end. While this is the *de-facto* standard for model validation, cross-validation still suffers from a key problem.

Quality Assurance of Artificial Intelligence Solutions: Validation

We need something else. As some of you might have heard already, cross-validation can come to rescue.

Set of known actions	Outcome

In cross-validation you split the dataset in n parts, called *folders*, and you validate the model using $n-1$ folders as training and the remaining folder as test. The process is then repeated so that each folder is used as test exactly once.

Typically, a 10-fold cross-validation is applied, with each folder defined randomly.

The initial random split of the folders might impact the evaluation procedure —> we might be (un)lucky with the split, biasing the results.

Dealing with randomness: just re-run the validation multiple times (e.g., 10 times) and analyze the results using statistics!

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Validation

This is an additional reason why experimenting with machine learning systems takes a lot: in the validation phase, you need to run expensive computation multiple times!

And the story is not yet at the end. While this is the *de-facto* standard for model validation, cross-validation still suffers from a key problem.

Quality Assurance of Artificial Intelligence Solutions: Validation

We need something else. As some of you might have heard already, cross-validation can come to rescue.

Set of known actions	Outcome

In cross-validation you split the dataset in n parts, called *folders*, and you validate the model using $n-1$ folders as training and the remaining folder as test. The process is then repeated so that each folder is used as test exactly once.

Typically, a 10-fold cross-validation is applied, with each folder defined randomly.

The initial random split of the folders might impact the evaluation procedure → we might be (un)lucky with the split, biasing the results.

Dealing with randomness: just re-run the validation multiple times (e.g., 10 times) and analyze the results using statistics!

Si chiama **assunzione IID**: **Indipendenti e Identicamente Distribuiti**, ed è alla base di quasi tutto ciò che funziona in Machine Learning e Statistica.

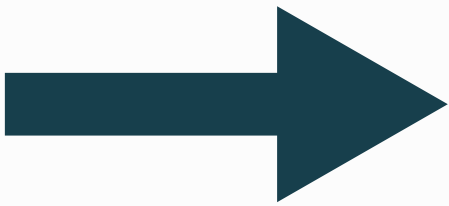
L'IID assume **due proprietà** distinte ma necessarie >>

17/18

4

12

datapizza Segui



Tutti le variabili in un campione devono essere:

- > **Indipendenti:**
Ogni dato nel set non influisce sugli altri, come conoscere un'osservazione non dice nulla sulle successive
- > **Tratte dalla stessa distribuzione:**
Tutti i dati seguono lo stesso modello o la stessa legge probabilistica (stessa media, stessa varianza, stessa forma)

17/18

4

12

datapizza Segui

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Validation

Re-running the validation will surely help addressing the randomness issue. But wait: there are some other problems.

Problem #1: Suppose you have a highly unbalanced problem. Independently from the re-running way of operating, you may still have issues when training models because of the lack of data —> in machine learning systems, if you do not have enough examples, you cannot learn!

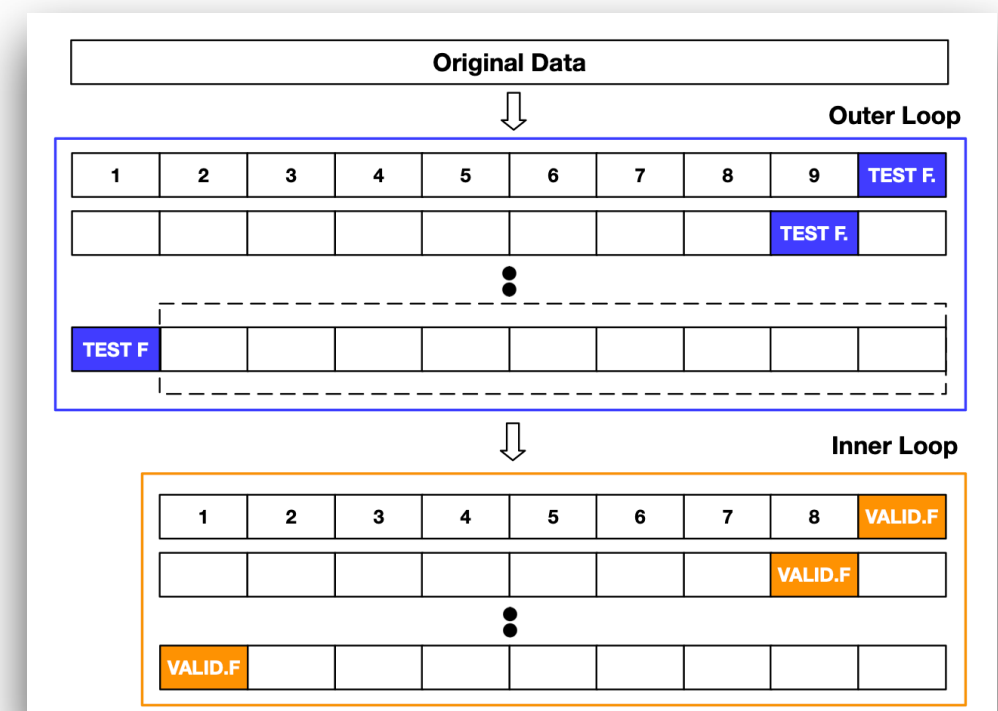
Solution: Perform a pseudo-random splitting of the folders. Rather than relying on randomness completely, you can apply a stratified sample approach where the examples of the minority class are equally distributed among the folders. In this way, you can still rely on seemingly real datasets!

Problem #2: To configure hyper-parameters of the learners, you might want to rely on an independent set to increase the degree of reality.

Solution: Cross-fold nested cross validation.

Split the entire set of data in n folds. One of these folds is retained as test and left untouched until the end of the computations done in the inner loop.

The remaining $n-1$ folds are instead used within the inner loop: one fold is reserved for the validation, while $n-2$ of them are used to train. You use the validation set to experiment with parameters, using an inner cross-validation.



Software Engineering for Artificial Intelligence: Validation and Verification

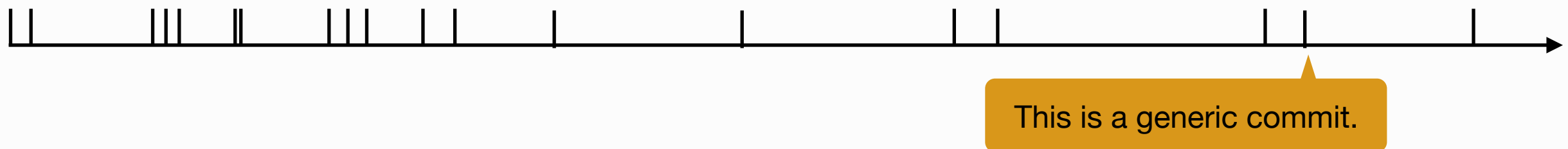
Validation and Verification of AI-Enabled Systems: Validation

Re-running the validation will surely help addressing the randomness issue. But wait: there are some other problems.

Problem #3: Suppose you have temporal data, namely you want to do predictions based on data that follow a temporal distribution.

Let's do a software engineering example. You want to use the previous commit data to predict the likelihood of a new change introducing a security vulnerability.

You have a bunch of commits to learn from, and you want to use them.



Machine learning means learning *from the past*. What happens if we apply a cross-validation here? Temporal data would not follow a temporal order anymore because of random splitting! This is a huge threat to reliability of the model validation process, since you may train the model with future data to predict past data, which is totally unrealistic! —> unfortunately, several research papers did this methodological error...

If you face a similar situation with cross-fold validation, you are falling into a case of *leaky validation*.

Data leakage. Issue arising when the machine learning model is able to work in an accurate manner at training time, but not at release time.

Software Engineering for Artificial Intelligence: Validation and Verification

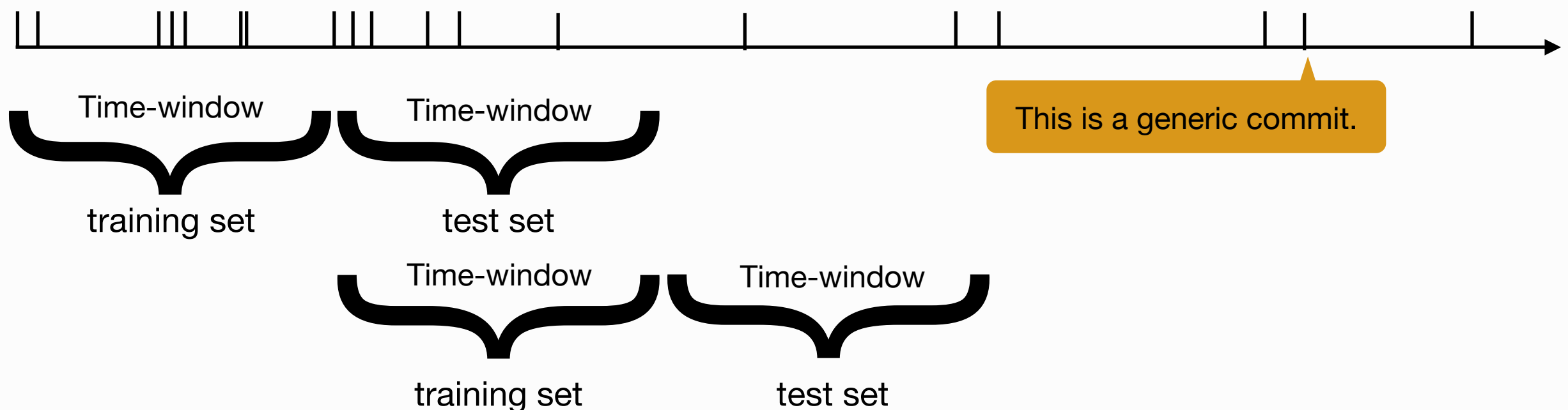
Validation and Verification of AI-Enabled Systems: Validation

Re-running the validation will surely help addressing the randomness issue. But wait: there are some other problems.

Problem #3: Suppose you have temporal data, namely you want to do predictions based on data that follow a temporal distribution.

Let's do a software engineering example. You want to use the previous commit data to predict the likelihood of a new change introducing a security vulnerability.

You have a bunch of commits to learn from, and you want to use them.



Hence, time-sensitive validation. You iteratively train on data of a specific time window and test on the subsequent time window. In this way you preserve temporal data and experiment with a more realistic use case. Yet, you have to tune the time window: this is strictly dependent on the problem.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Validation

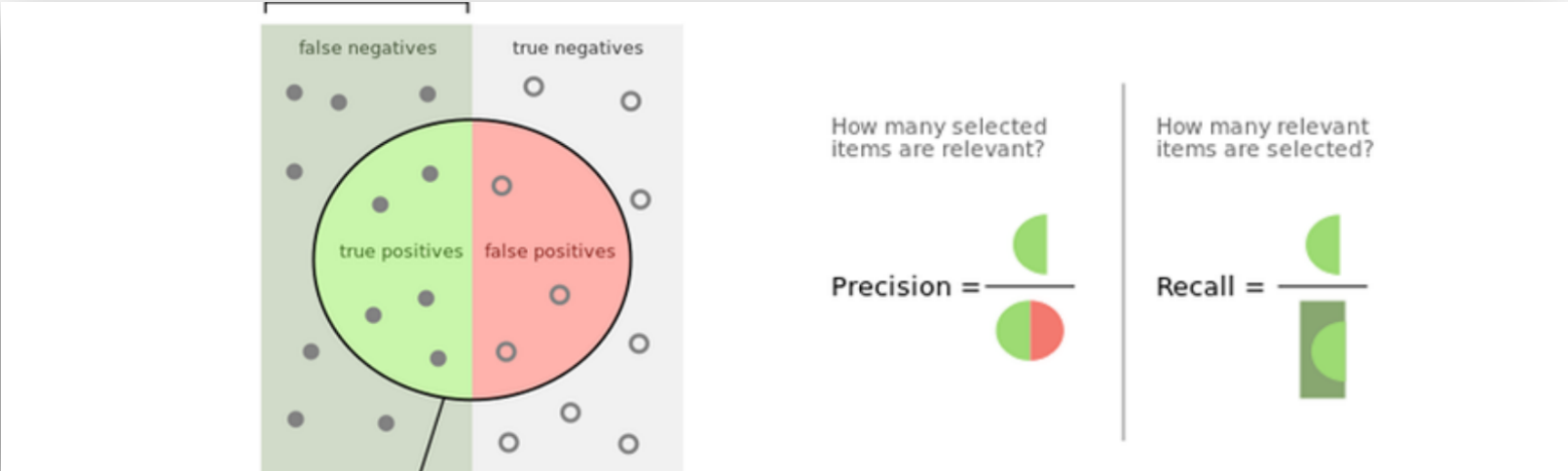
The validation strategy must, however, be completed with some quantitative evaluation of the performance of the artificial intelligence technique experimented.

	True positives	True negatives
Predicted true positives	True positives	False positives
Predicted true negatives	False negatives	True negatives

Validation means comparing the predictions made with the actual oracle. Hence, you can compute the so-called confusion matrix.

It summarizes the number of (1) truly predicted instances of the positive class, i.e., true positives, (2) wrongly predicted instances of the positive class, i.e., false positives, (3) truly predicted instances of the negative class, i.e., true negatives, (4) wrongly predicted instances of the positive class, i.e., false negatives.

On the basis of the confusion matrix, we can compute a number of metrics.



Precision and recall are among the most widely used metrics for model assessment. Yet, they do not depict the whole picture.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Validation

The validation strategy must, however, be completed with some quantitative evaluation of the performance of the artificial intelligence technique experimented.

	True positives	True negatives
Predicted true positives	True positives	False positives
Predicted true negatives	False negatives	True negatives

Validation means comparing the predictions made with the actual oracle. Hence, you can compute the so-called confusion matrix.

It summarizes the number of (1) truly predicted instances of the positive class, i.e., true positives, (2) wrongly predicted instances of the positive class, i.e., false positives, (3) truly predicted instances of the negative class, i.e., true negatives, (4) wrongly predicted instances of the positive class, i.e., false negatives.

On the basis of the confusion matrix, we can compute a number of metrics.

$$Precision = \frac{TP}{(TP + FP)}$$

$$Recall = \frac{TP}{(TP + FN)}$$

$$Specificity = \frac{TN}{(TN + FP)}$$

$$Accuracy = \frac{(TP + TN)}{(TP + TN + FP + FN)}$$

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP) \cdot (TP + FN) \cdot (TN + FP) \cdot (TN + FN)}}$$

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Validation

The validation strategy must, however, be completed with some quantitative evaluation of the performance of the artificial intelligence technique experimented.

	True positives	True negatives
Predicted true positives	True positives	False positives
Predicted true negatives	False negatives	True negatives

Validation means comparing the predictions made with the actual oracle. Hence, you can compute the so-called confusion matrix.

It summarizes the number of (1) truly predicted instances of the positive class, i.e., true positives, (2) wrongly predicted instances of the positive class, i.e., false positives, (3) truly predicted instances of the negative class, i.e., true negatives, (4) wrongly predicted instances of the positive class, i.e., false negatives.

On the basis of the confusion matrix, we can compute a number of metrics.

Precision = $\frac{TP}{TP + FN}$

Specificity = $\frac{TN}{TN + FP}$

Accuracy = $\frac{TP + TN}{TP + TN + FP + FN}$

MCC = $\frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP) \cdot (TP + FN) \cdot (TN + FP) \cdot (TN + FN)}}$

Be careful to accuracy! You may obtain a 99% accuracy without any true positive!

Validation and Verification of AI-Enabled Systems: Validation

However, be careful: metrics are “just” metrics! Unfortunately, it is quite common that accuracy reported from offline evaluations with test data is overly optimistic.

We discuss two common reasons for this discrepancy, to provide awareness of pitfalls and to encourage developers involved in model development and evaluation to pay particular attention to avoid these pitfalls.

- 1 *Using test data that are not representative.* We usually assume that the training and test data are representative of the data to be expected in production. Unfortunately, this is not always true in practice.

For example, we may develop a cancer prediction model on scans from equipment in a specific hospital with patients typical for that hospital. While the model may work well for predictions in this specific setting, we may see much lower prediction accuracy when deploying in a different hospital, because we are trying to generalize beyond the training distribution.

When the distribution of input data in production is different from the distribution during training, we expect many **out-of-distribution predictions** for which the accuracy may be much lower than reported from in-distribution test data.

Unfortunately, collecting representative training and test data for the target distribution of a production system can be difficult and expensive. That’s why **data requirements** have become so important.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Validation

However, be careful: metrics are “just” metrics! Unfortunately, it is quite common that accuracy reported from offline evaluations with test data is overly optimistic.

We discuss two common reasons for this discrepancy, to provide awareness of pitfalls and to encourage developers involved in model development and evaluation to pay particular attention to avoid these pitfalls.

assumes that the training
data is collected in production.

model on scans from
that hospital. While the
distribution of test data at that hospital, because we are

different from the distribution
of test data. **predictions** for which the
distribution test data.

test data for the target
is more expensive. That's why

Dissecting racial bias in an algorithm used to manage the health of populations

ZIAD OBERMEYER , BRIAN POWERS, CHRISTINE VOGELI, AND SENDHIL MULLAINATHAN  [Authors Info & Affiliations](#)

SCIENCE • 25 Oct 2019 • Vol 366, Issue 6464 • pp. 447-453 • DOI: 10.1126/science.aax2342

↓ 178,441 ” 4,302



 CHECK ACCESS

Racial bias in health algorithms

The U.S. health care system uses commercial algorithms to guide health decisions. Obermeyer *et al.* find evidence of racial bias in one widely used algorithm, such that Black patients assigned the same level of risk by the algorithm are sicker than White patients (see the Perspective by Benjamin). The authors estimated that this racial bias reduces the number of Black patients identified for extra care by more than half. Bias occurs because the algorithm uses health costs as a proxy for health needs. Less money is spent on Black patients who have the same level of need, and the algorithm thus falsely concludes that Black patients are healthier than equally sick White patients. Reformulating the algorithm so that it no longer uses costs as a proxy for needs eliminates the racial bias in predicting who needs extra care.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Validation

However, be careful: metrics are “just” metrics! Unfortunately, it is quite common that accuracy reported from offline evaluations with test data is overly optimistic.

We discuss two common reasons for this discrepancy, to provide awareness of pitfalls and to encourage developers involved in model development and evaluation to pay particular attention to avoid these pitfalls.

assumes that the training
data is representative of the data
seen in production.

model on scans from
a different hospital. While the
model is performing well on the
training data, we may see much
poorer performance on the test data
from that hospital, because we are

different from the distribution
of the training data. **Predictions**
for which the model has no
distribution test data.

test data for the target
population is expensive. That's why

Women in Clinical Trials – Overcoming a History of Bias and Exclusion

Healthcare should be tailored to everyone, yet for decades medical research has largely overlooked nearly half of the global population. As of February 2025, women make up 49.6% of the world's population¹, and according to the United Nations, this sex ratio is expected to become even more balanced by 2050². On average women tend to live longer than men, though the life expectancy varies by country.

Women also access healthcare services more frequently and account for significantly higher share of health expenditures in comparison to men. Yet up until the early 1990s - a period during which many drugs still in use today were approved - women faced significant underrepresentation in clinical trials. This lack of inclusion has resulted in critical gaps in our understanding of how particular treatments impact women, increasing the risk of adverse effects and limiting the effectiveness of medical interventions tailored to their needs.

Despite making up almost half of the world's population, women have historically been misrepresented in clinical studies. For decades, medical research conducted primarily on men has been used to draw insights into diseases, drug efficacy and medical devices - often applying these findings to women without sufficient evidence. However, a woman's anatomy, genetics, hormonal balances, and organ systems differ significantly from those of men. Given these differences, it is neither accurate nor appropriate to assume that treatments studied predominantly in men will have the same effects on women.

Validation and Verification of AI-Enabled Systems: Validation

However, be careful: metrics are “just” metrics! Unfortunately, it is quite common that accuracy reported from offline evaluations with test data is overly optimistic.

We discuss two common reasons for this discrepancy, to provide awareness of pitfalls and to encourage developers involved in model development and evaluation to pay particular attention to avoid these pitfalls.

2

Label leakage. With label leakage, the model may not learn the actual concepts of the tasks (e.g., detect cancer in a scan) but instead picks up on subtle shortcuts in the data that leak the true label but may not generalize in practice. The best-known example of label leakage is probably the frequently shared [urban legend](#) of an early military AI project trying to detect Russian tanks in pictures with neural networks.

The model accurately predicted tanks in test data but utterly failed in production. It turned out that the data was gathered so that **pictures with tanks were taken on sunny days**, whereas the **other pictures on cloudy days**. The model then simply learned to distinguish brightness or weather. The correlation of weather to the label is a shortcut that existed in both training and test data, but it obviously is not something we can rely on in production.

Label leakage can be challenging to detect. Beyond being suspicious of “too good to be true” results, a good strategy is to use **explainability techniques**.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Validation

Independently from the validation strategy you decide to use, you should never forget of the two main training concerns governing machine learning.

Overfitting: Situation arising when a machine learner learns *too well* the training data. More particularly, the noise or random fluctuations in the training data are learned as concepts by the model: these concepts do not apply to new data and negatively impact the models ability to generalize. Hence, you have good performance on the training data and poor performance on the test data. The use of cross-validation helps you assessing the model for overfitting: you have the chance to evaluate the learner on multiple unseen set of data and estimate its goodness in various situations. Be careful to model configuration here. Configuring hyper-parameters is good, but it may occasionally increase the chances of overfitting: the more hyper-parameters you set, the more the learner will be able to adapt itself to the training data, hence possibly overfitting.

Underfitting: Situation arising when a machine learner is not able to learn training data well and, for this reason, is not able to generalize to new data.

Hence, you have poor performance on the training data and poor performance on the test data.

Underfitting it typically way less discussed than overfitting because it can easily be assessed by computing metrics. The remedy is to move on and try alternate machine learning algorithms.

In any case, these concepts emphasize the need for experiments to assess the goodness of machine learning systems. Multiple model validation is therefore necessary... but unfortunately not sufficient!

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Verification

Model validation is good, as it allows data scientists and software engineers to understand how good the learning worked. Model validation is bad, as it *starts from the assumption that the training data meets the requirements!*

As a matter of fact, you might obtain high prediction performance without meeting any of the requirements: you just need to feed the machine learner as you like!

In traditional software engineering, you have a similar problem: you can design test cases solely based on what you implemented (BTW, white-box testing) without considering what you promised, i.e. the requirements.

Similarly to traditional software engineering, you need to complement white-box testing with black-box testing, i.e., the verification of the system solely based on its specifications.

Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

Model testing

Model level tests consider an ML model in isolation, e.g., without taking any of the other components of the MLS into account.

Integration testing

Testing in which software components, hardware components, or both are combined and tested to evaluate the interaction between them.

System testing

Testing conducted on a complete, integrated ML based system to evaluate the system's compliance with its specified requirements.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Verification

Model validation is good, as it allows data scientists and software engineers to understand how good the learning worked. Model validation is bad, as it *starts from the assumption that the training data meets the requirements!*

As a matter of fact, you might obtain high prediction performance without meeting any of the requirements: you just need to feed the machine learner as you like!

In traditional software engineering, you have a similar problem: you can design test cases solely based on what you implemented (BTW, white-box testing) without considering what you promised, i.e. the requirements.

Similarly to traditional software engineering, you need to complement white-box testing with black-box testing, i.e., the verification of the system solely based on its specifications.

Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

Model testing

Model testing is what we can do with validation, but as you can see, it is just a small part of the problem! This is what we can do with validation, but as you can see, it is just a small part of the problem! This is what we can do with validation, but as you can see, it is just a small part of the problem!

Integration testing

Testing in which software components, hardware components, or both are combined and tested to evaluate the interaction between them.

System testing

Testing conducted on a complete, integrated ML based system to evaluate the system's compliance with its specified requirements.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Verification

Model validation is good, as it allows data scientists and software engineers to understand how good the learning worked. Model validation is bad, as it *starts from the assumption that the training data meets the requirements!*

As a matter of fact, you might obtain high prediction performance without meeting any of the requirements: you just need to feed the machine learner as you like!

In traditional software engineering, you have a similar problem: you can design test cases solely based on what you implemented (BTW, white-box testing) without considering what you promised, i.e. the requirements.

Similarly to traditional software engineering, you need to complement white-box testing with black-box testing, i.e., the verification of the system solely based on its specifications.

Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

Model testing

Model testing is what we can do with validation, but as you can see, it is just a small part of the problem! Validation, e.g., without taking requirements into account.

Integration testing

Testing in which software components, hardware components, or both are integrated and tested for proper interaction between them.

System testing

These types of testing approaches are similar to what you use for traditional systems: you verify the system as a whole, often involving stakeholders. Test the ML based system to evaluate the system's compliance with its specified requirements.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Verification

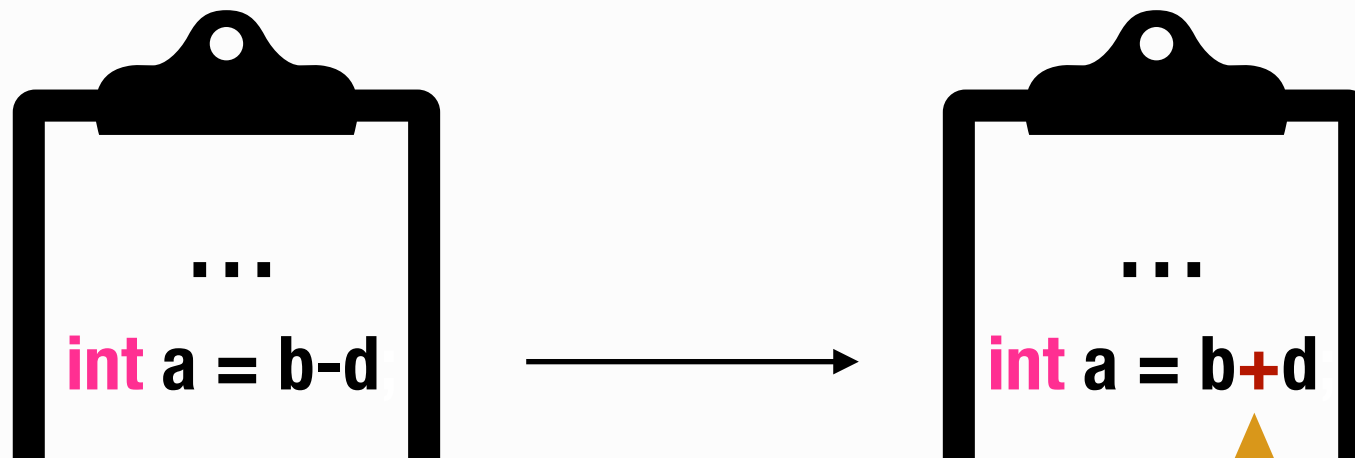
Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

The general idea of input testing is to identify a set of input data similar to the inputs taken by the machine learning solution to be tested and verify how it behaves.

Nonetheless, there is the usual problem: where to find similar data before releasing the machine learner?

Sure. Let's borrow ideas from mutation testing!



In mutation testing, you create *mutants*, namely artificially-modified compilable versions of the original source code, that simulate possible bugs arising in reality.

The goal of mutation testing is to assess the goodness of test cases in catching mutations, i.e., the assumption is that mutations are valid substitute of real bugs.

Empirical Software Engineering (2020) 25:5193–5254
<https://doi.org/10.1007/s10664-020-09881-0>



Testing machine learning based systems: a systematic mapping

Vincenzo Riccio¹ · Gunel Jahangirova¹ · Andrea Stocco¹ · Nargiz Humbatova¹ · Michael Weiss¹ · Paolo Tonella¹

Published online: 15 September 2020
© The Author(s) 2020

Abstract

Context: A Machine Learning based System (MLS) is a software system including one or more components that learn how to perform a task from a given data set. The increasing adoption of MLSs in safety critical domains such as autonomous driving, healthcare, and finance has fostered much attention towards the quality assurance of such systems. Despite the advances in software testing, MLSs bring novel and unprecedented challenges, since their behaviour is defined jointly by the code that implements them and the data used for training them.

Objective: To identify the existing solutions for functional testing of MLSs, and classify them from three different perspectives: (1) the context of the problem they address, (2) their features, and (3) their empirical evaluation. To report demographic information about the ongoing research. To identify open challenges for future research.

Method: We conducted a systematic mapping study about testing techniques for MLSs driven by 33 research questions. We followed existing guidelines when defining our research protocol so as to increase the repeatability and reliability of our results.

Results: We identified 70 relevant primary studies, mostly published in the last years. We identified 11 problems addressed in the literature. We investigated multiple aspects of the testing approaches, such as the used/proposed adequacy criteria, the algorithms for test input generation, and the test oracles.

Conclusions: The most active research areas in MLS testing address automated scenario/input generation and test oracle creation. MLS testing is a rapidly growing and developing research area, with many open challenges, such as the generation of realistic inputs and the definition of reliable evaluation metrics and benchmarks.

Keywords Systematic mapping · Systematic review · Software testing · Machine learning

Communicated by: David Lo

✉ Vincenzo Riccio
vincenzo.riccio@usi.ch

Extended author information available on the last page of the article.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Verification

Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

The power of this method stands in mutating the inputs taken by the system to simulate real-world conditions that may impact the performance of the machine learner.

In this way, a machine learning algorithm can be experimented with inputs similar to those expected in production, hence mimicking a realistic *in-vivo* evaluation.

This is still an active research problem and, therefore, only a few examples are available.

The problem of image classification is one of the most widely investigated: not only because images can be useful to solve a variety of real-world problems, but also because they are computationally hard to treat and, therefore, are valuable use cases.

In this case, input mutations pertain to changes to pixel values or cropping. Recent work has also attempted to automate the generation of mutations able to simulate environmental conditions such as rain, fog, snow, and occlusion created by mud/snow on the camera.

Input mutations have been mainly experimented on deep learning systems: in this case, input mutations can be generated so that the neuron coverage of the deep networks is maximized.

So, the problem of test input data generation can be expressed as an optimization problem that aims at generating realistic cases while maximizing the coverage of the neural network.

All good and nice, but there is a missing piece: if we want to do testing, we do not only need input mutations but also the corresponding *oracles*!

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Verification

Input testing

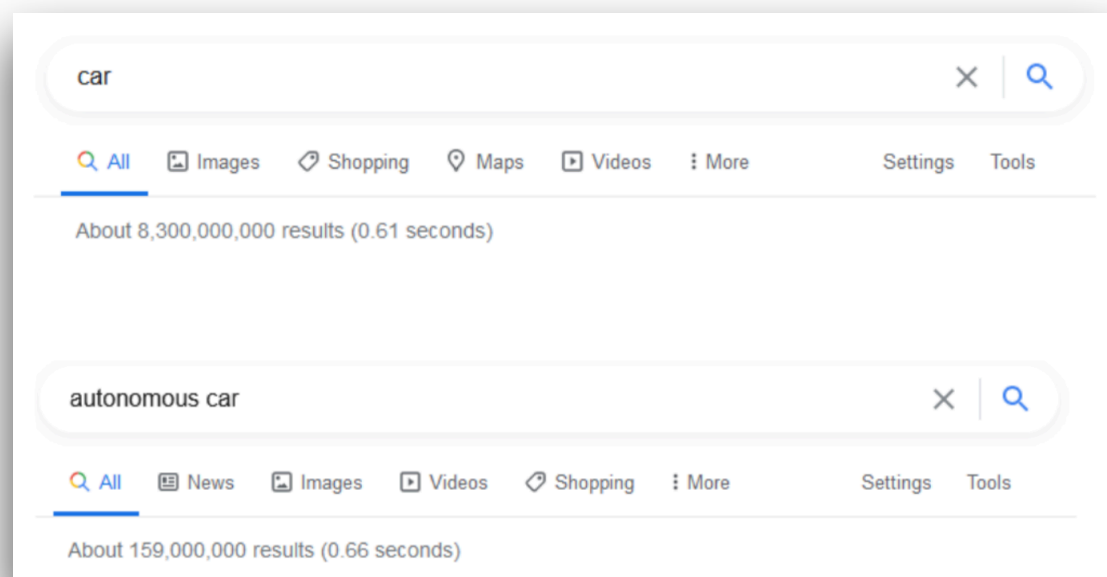
Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

In other terms, for each input mutation we want to have an oracle reporting the expected behavior of the system, so that we can verify if the system reacts as it should.

The most recent trend is represented by the use of *metamorphic testing*: it describes the system functionality in terms of generic relations between inputs, the generic transformations of those inputs and their outputs, rather than as mappings of specific inputs to specific outputs.

—> Instead of clearly defined inputs and logical flows based on explicit programming statements, a ML-based system is based on potentially huge input spaces with probabilistic outcomes from largely black-box components (models). That is why you cannot map specific inputs to outputs and need relations.

The typical example used for metamorphic testing comes from the testing of search engines.



Suppose you want to verify the behavior of Google in response to these two queries.

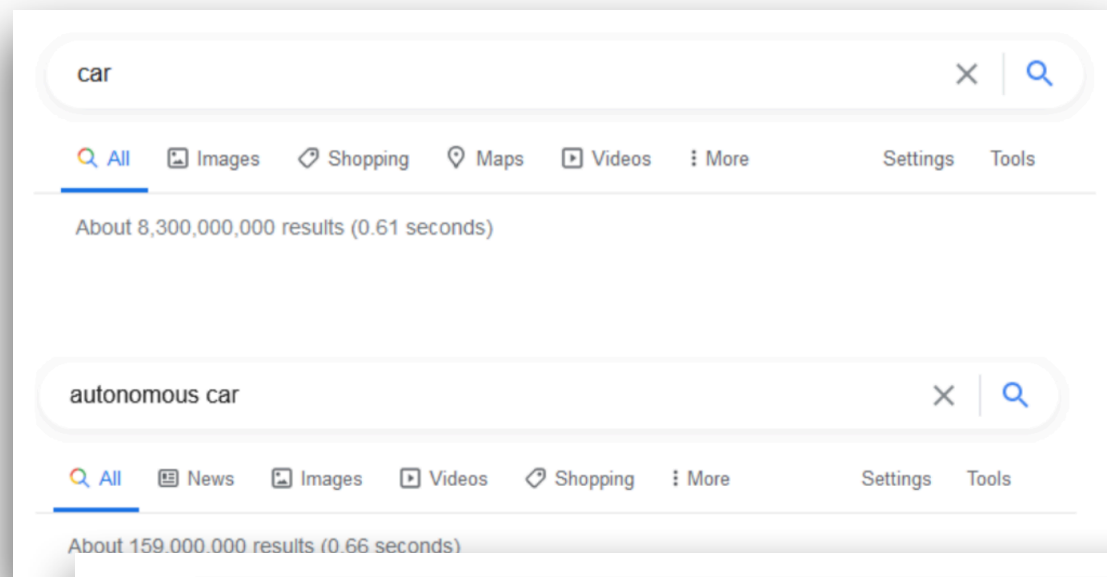
The first query is “*car*”. The second query adds another word to the first query, “*autonomous*”. This addition of a restrictive search keyword is an example of an input transformation (or “morphing”, in the spirit or *metamorphic* testing).

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Verification

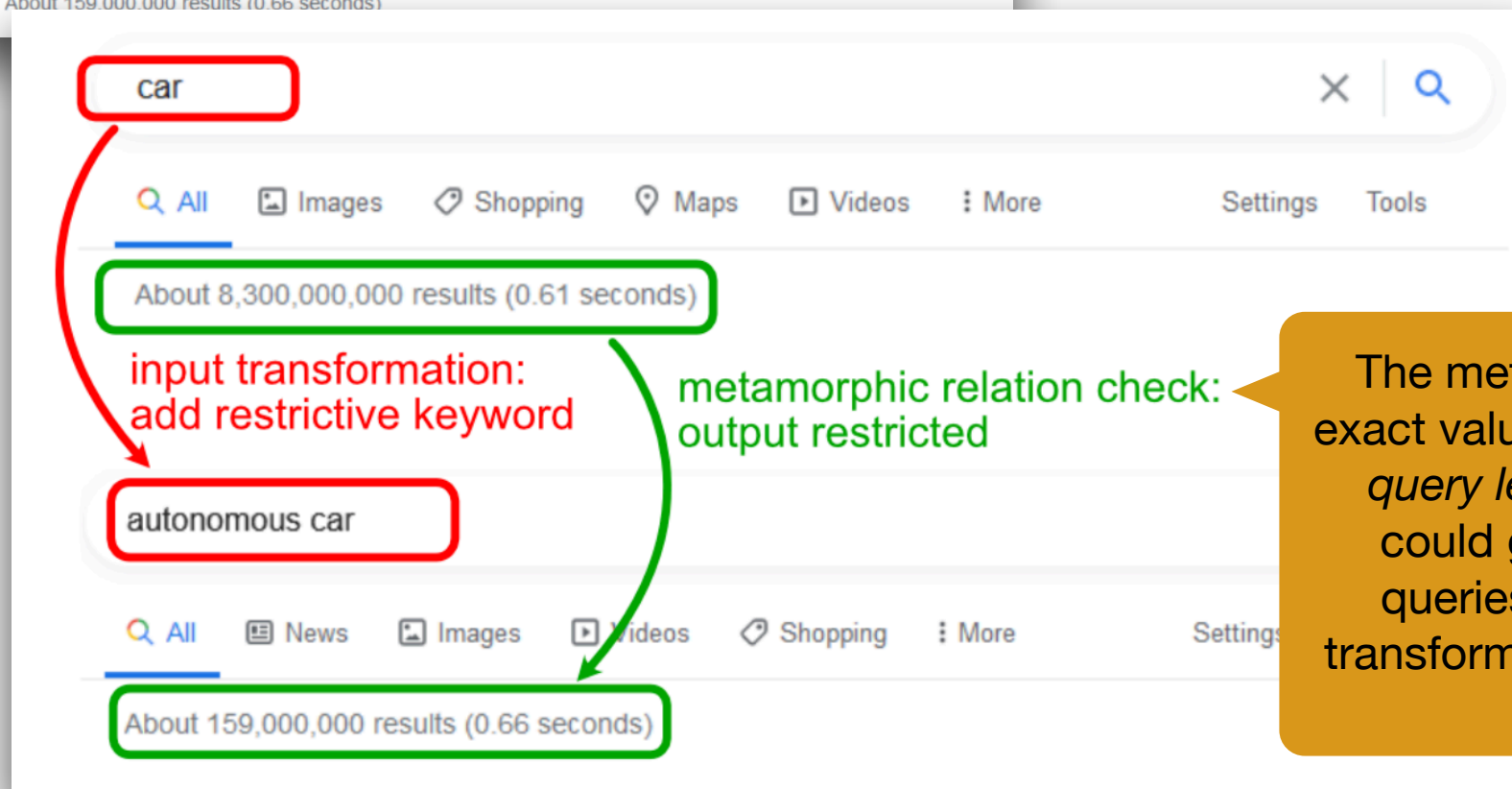
Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.



For each input transformation, metamorphic testing defines a matching relation that should hold when the input transformation is applied.

In the example, we expect to find less results when executing the more restrictive query. This expectation is the relation that we can specify.



The metamorphic test would not specify these exact values, but rather the relation “*restricting the query leads to fewer search results*”. And one could generate several (seed) inputs (search queries), associated restrictive keywords for transformations, and run the query and check the metamorphic relation holds.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Verification

Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

Based on this example, let's define the key terms more formally.

Morph/Transform. An input modification/mutation created ad-hoc and so that the defined metamorphic relations should hold.

Metamorphic Relation. The defined transformation of the input should have a known/measurable effect on the output. Checking that this relation holds after the transformation is the *test oracle* of metamorphic testing.

Seed Inputs. The inputs that are used as initial inputs for the tests, to be transformed. If you know the output of the seed input, you may use it to define a stricter relation (output should be correct). But even without the seed output, you can still define a relation check, but it might be a bit more relaxed (output should be similar but you don't know if it is correct).

Metamorphic testing has been defined in 1998 for traditional software systems. While this form of testing was mainly a research theme with few practical applications, this seems to be quite useful when testing machine learning systems.

Let's contextualize metamorphic testing in a key current problem of software engineering for artificial intelligence, i.e., the testing of self-driving cars.

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Verification

Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

Metamorphic testing of self-driving cars.

Morph/Transform

Moving, tilting, blurring, scaling, zooming, adding fog, adding rain, etc.

Metamorphic relations

The autonomous driving decisions should show minimal divergence on the same input images after the transformations.

Seed inputs

Real images from car cameras.



Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Verification

Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

Metamorphic testing of self-driving cars.

Morph/Transform

Moving, tilting, blurring, scaling, zooming, adding fog, adding rain, etc.

Metamorphic relations

The autonomous driving decisions should show minimal divergence on the same input images after the transformations.

Seed inputs

Real images from car cameras



The test can be considered successful if the behavior of the machine learner is similar enough to the original case, i.e., if the system proposes a very similar driving path, with minimal differences across all transformations on acceleration, steering, etc.



Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems: Verification

Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

Metamorphic testing of self-driving cars.

Morph/Transform

Moving, tilting, blurring, scaling, zooming, adding fog, adding rain, etc.

Metamorphic relations

The autonomous driving decisions should show minimal divergence on the same input images after the transformations.

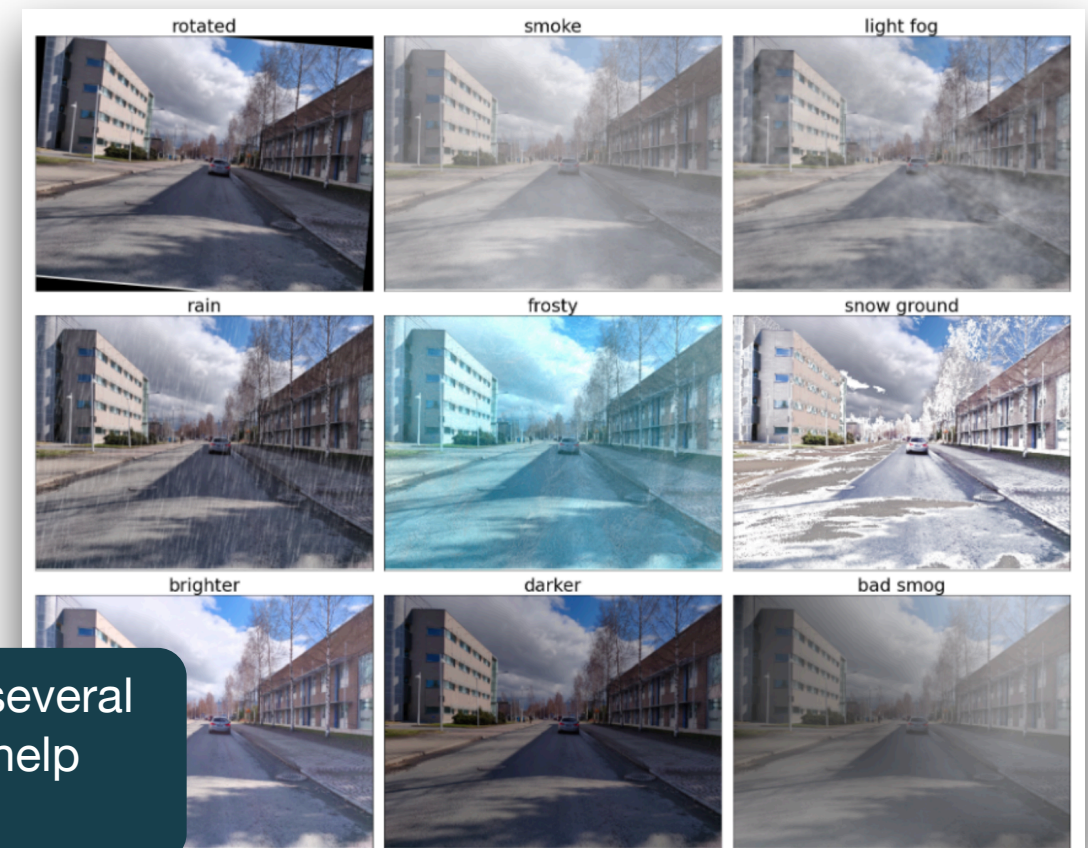
Seed inputs

Real images from car cameras.

More complex input mutations might further increase the degree of realism. For instance, *Generative Adversarial Networks* can be used to produce realistic scenarios that also simulate the presence of objects in the image.

The higher the complexity of the mutations, the higher the confidence that the system will work in practice.

Automated transformation tools can be used. There are several libraries for image augmentation, originally created to help increase training dataset sizes in machine learning.



Software Engineering for Artificial Intelligence: Verification and Validation

Before leaving... What Should You Remember?

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems

To *verify* that the implementation of *next_date* implements the specification for that problem **correctly**, we can write tests that check input-output pairs. If any test case fails, either the test was wrong, or we have found a bug.

To *validate* that the implementation **meets a customer's needs**, we might show the finished product to the customer. If the customer needed a *next_date* function in the Hebrew calendar instead, we simply implemented the wrong specification.

In that case, the implementation would still be *technically correct according to the provided specification*, but it would simply not be useful to the customer.

The correctness of software is primarily evaluated through **testing**. We can not exhaustively test all possible inputs, because there are simply too many.

So, while a good testing regime might give us some confidence in correctness, it will not provide correctness guarantees —we may miss some bugs in testing.

```
graph TD
    A[Problem and needs] --> B[Requirements/specification]
    B --> C[Design & coding]
    C --> D[Implementation]
    D --> E[Verification "correct"]
    E --> B
    B --> F[Validation "useful"]
    F --> A
```

Software Engineering for Artificial Intelligence: Validation and Verification

Validation and Verification of AI-Enabled Systems

How we evaluate AI models differs fundamentally from traditional software testing. Generally speaking, AI is just an algorithm that takes inputs to compute an output; the key difference is that it has been learned from data rather than manually coded.

Again, the question is, “*Correct with regard to what?*”

For AI models, we have **no specification** that we could use to determine which outputs are correct for any given input. **After all, we use AI precisely because we do not have such specifications**, typically because the problem is too complex to specify or the rules are simply unknown.

We might have *ideas, goals, intuitions*, and *general descriptions*, for example “detect whether there is cancer in the image”, as well as examples of input-output pairs, which could be seen as partial or implicit specifications, but *nothing we can write down* as concrete rules as to when an output is correct for any given input.

In practice, we fundamentally accept that we cannot avoid some wrong predictions. When using AI, *we cannot* and *do not expect* all outputs to be correct, if we can even determine whether a prediction is correct in the first place. We would not write a test suite with individual input-output examples and expect all tests to pass. A single wrong prediction is not a reason to reject a model.

Overall, we have a **weak notion of correctness**. We evaluate models with examples, but not against specifications of what the model should generally do.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Validation

Independently from the presence of technical debt, a key and unavoidable quality assurance steps pertains to the validation of the model performance.

This typically starts with a so-called *in-vitro* experimentation of the capabilities of artificial intelligence solutions, namely the definition of an empirical setting where the accuracy of a model is experimented.

```
graph LR
    A[Set of known actions  
Barking during night.  
Wagging tail when happy.  
...  
Cross the road alone.] -- used to train --> B[Machine learning algorithm]
    B -- experimented on --> C[Set of unseen actions  
Barking during day.  
Hiding tail when sad.  
...  
Cross the road.]
    C --> D[Outcome]
```

As labeling additional data may be complex, the general idea is to split the original dataset into two sets, so that part of that is used for training and the remaining part for testing. NB: You cannot use the entire dataset for both training and testing!

Be careful here: if we want to see how effective a machine learning is, we need an *oracle* reporting the actual labels for the examples in the test set. The oracle will be only used to compare the predictions done by the learner with the truth values.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Validation

Re-running the validation will surely help addressing the randomness issue. But wait: there are some other problems.

Problem #1: Suppose you have a highly unbalanced problem. Independently from the re-running way of operating, you may still have issues when training models because of the lack of data → in machine learning systems, if you do not have enough examples, you cannot learn!

Solution: Perform a pseudo-random splitting of the folders. Rather than relying on randomness completely, you can apply a stratified sample approach where the examples of the minority class are equally distributed among the folders. In this way, you can still rely on seemingly real datasets!

Problem #2: To configure hyper-parameters of the learners, you might want to rely on an independent set to increase the degree of reality.

Solution: Cross-fold nested cross validation.

```
graph TD
    A[Original Data] --> B[Outer Loop]
    B --> C[Inner Loop]
    C --> D[Validation]
```

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Verification

Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

The general idea of input testing is to identify a set of input data similar to the inputs taken by the machine learning solution to be tested and verify how it behaves.

Nonetheless, there is the usual problem: where to find similar data before releasing the machine learner?

Sure. Let's borrow ideas from mutation testing!

```
graph LR
    A["int a = b - d"] --> B["int a = b + d"]
```

In mutation testing, you create *mutants*, namely artificially-modified compilable versions of the original source code, that simulate possible bugs arising in reality.

The goal of mutation testing is to assess the goodness of test cases in catching mutations, i.e., the assumption is that mutations are valid substitute of real bugs.

Abstract: Machine Learning based Systems (MLBS) is a software system including one or more components that learn from a previous task to solve a given task. The increasing adoption of MLBS in safety critical domains such as autonomous driving, healthcare, and financial has increased the need for systematic approaches for quality assurance of such systems. Despite the lack of research in machine learning, MLBS testing and quality assurance challenges, there has been a growing interest in the community to develop systematic approaches for testing MLBS. This paper presents a systematic mapping study of the existing research on MLBS testing and quality assurance. The study aims to identify the research gaps and provide a systematic overview of the current state of the art in MLBS testing and quality assurance. The study also identifies the research gaps and provides a systematic overview of the current state of the art in MLBS testing and quality assurance.

Software Engineering for Artificial Intelligence: Quality Assurance

Quality Assurance of Artificial Intelligence Solutions: Verification

Input testing

Input level tests analyze the training data used to train the ML components as well as the input data used at prediction time.

For each input transformation, metamorphic testing defines a matching relation that should hold when the input transformation is applied.

In the example, we expect to find less results when executing the more restrictive query. This expectation is the relation that we can specify.

input transformation: add restrictive keyword

metamorphic relation check: output restricted

The metamorphic test would not specify these exact values, but rather the relation “restricting the query leads to fewer search results”. And one could generate several (seed) inputs (search queries), associated restrictive keywords for transformations, and run the query and check the metamorphic relation holds.

Software Engineering for Artificial Intelligence

Validation and Verification

